

BIOEN 468/568 2022

Lab 6 Pre-lab

*Three Systematic Approaches to Derive Logical
Expressions for Finite-State Machines*

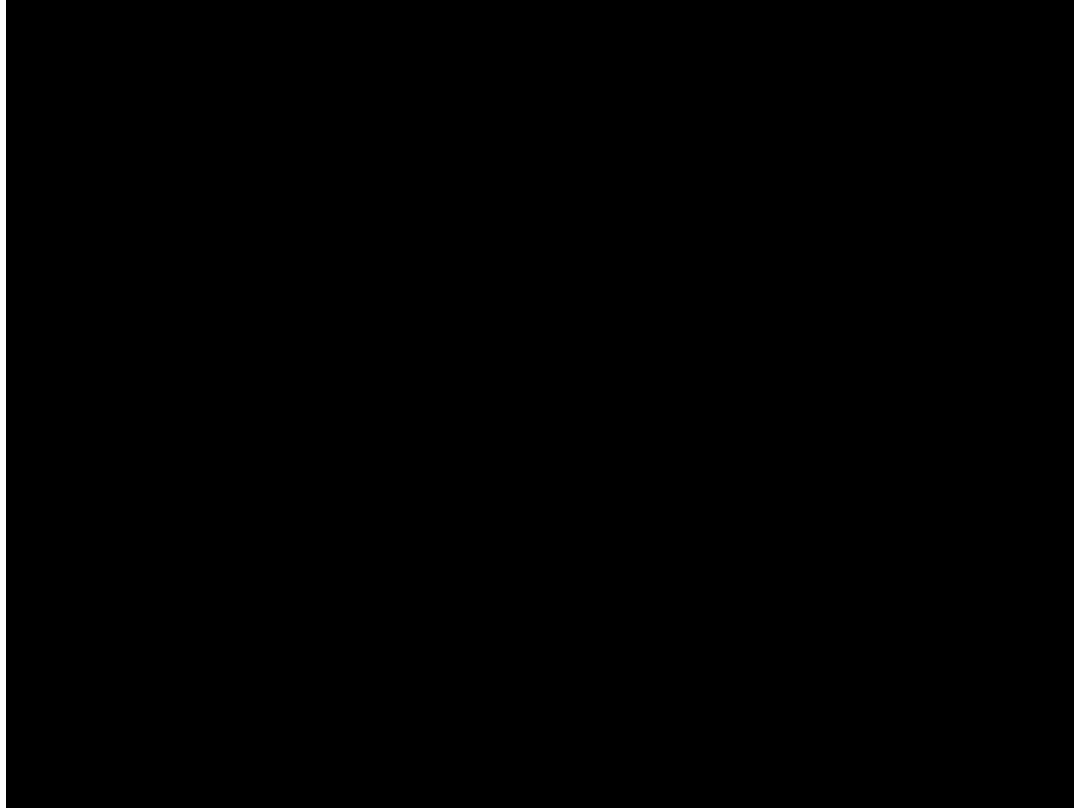
Presented by TA Tony Fu

Software specifications

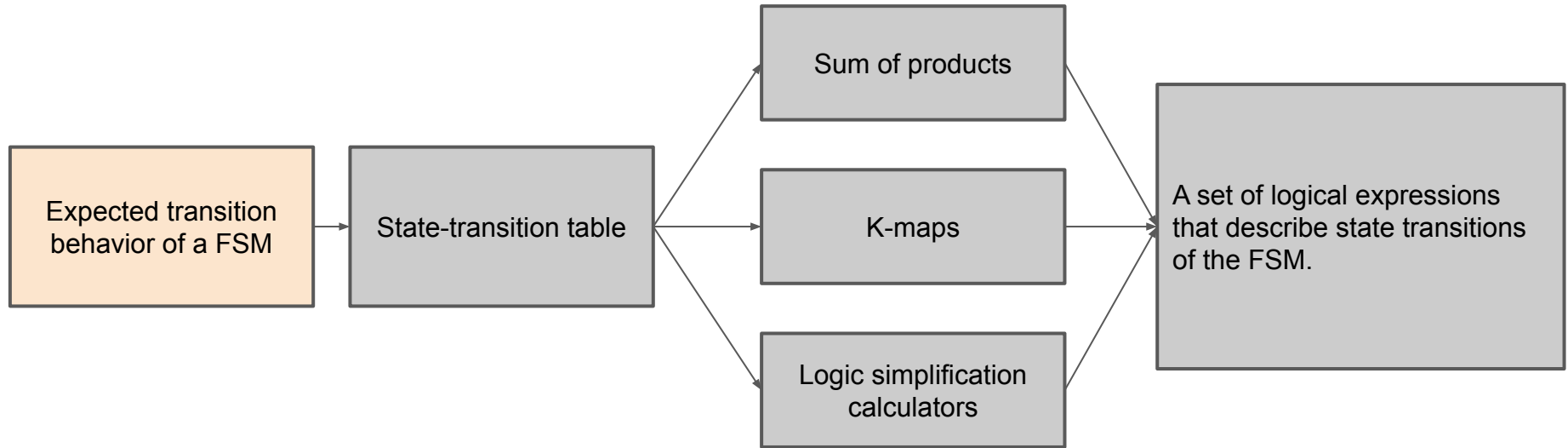
The controller is based on logical statements that determine the new state of each output based on the direction of rotation and the existing state of each LED. We will label the four outputs A, B, C and D, such that the sequence is A-B-C-D-A-B-C... to rotate one way, and D-C-B-A-D-C-B... to rotate the other way. In order to make the motor operate more smoothly, we may include some “handoff” time when two succeeding coils are energized. In this case, the sequence goes

$A \rightarrow AB \rightarrow B \rightarrow BC \rightarrow C \rightarrow CA \rightarrow A$ etc. *or* $C \rightarrow CB \rightarrow B \rightarrow BA \rightarrow A \rightarrow AC \rightarrow C$ etc.

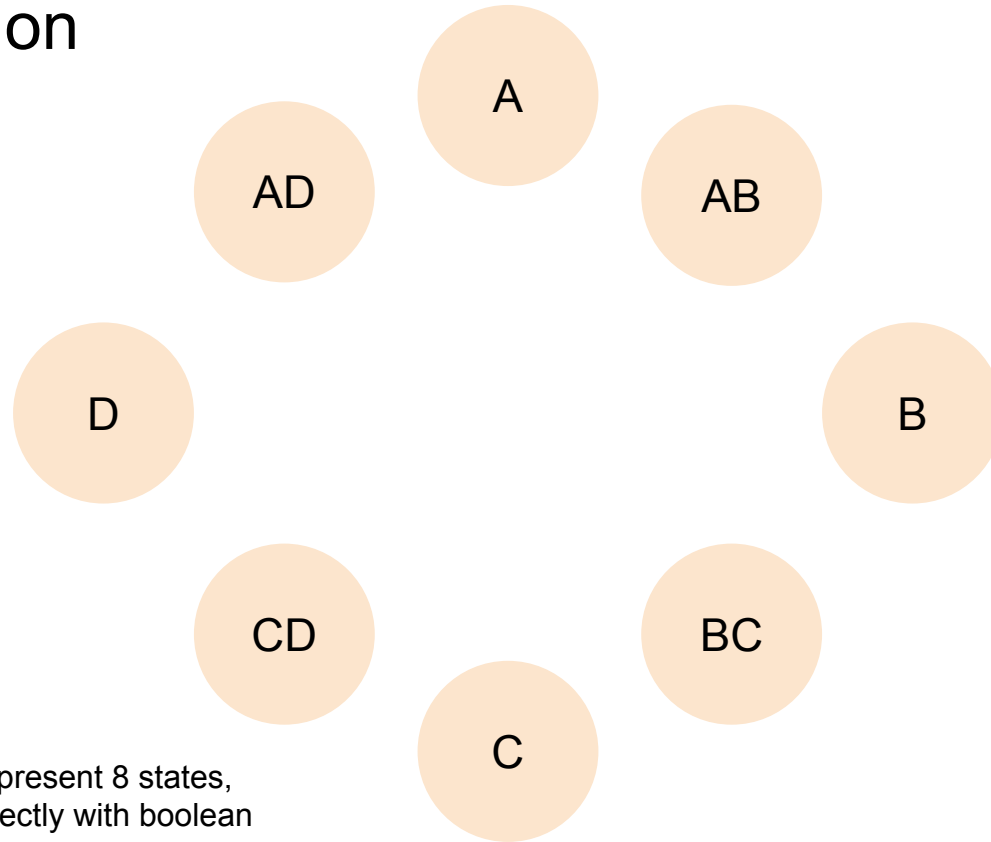
Pre-lab programming: Expected behavior



Systematic approaches to derive Finite State Machine expressions

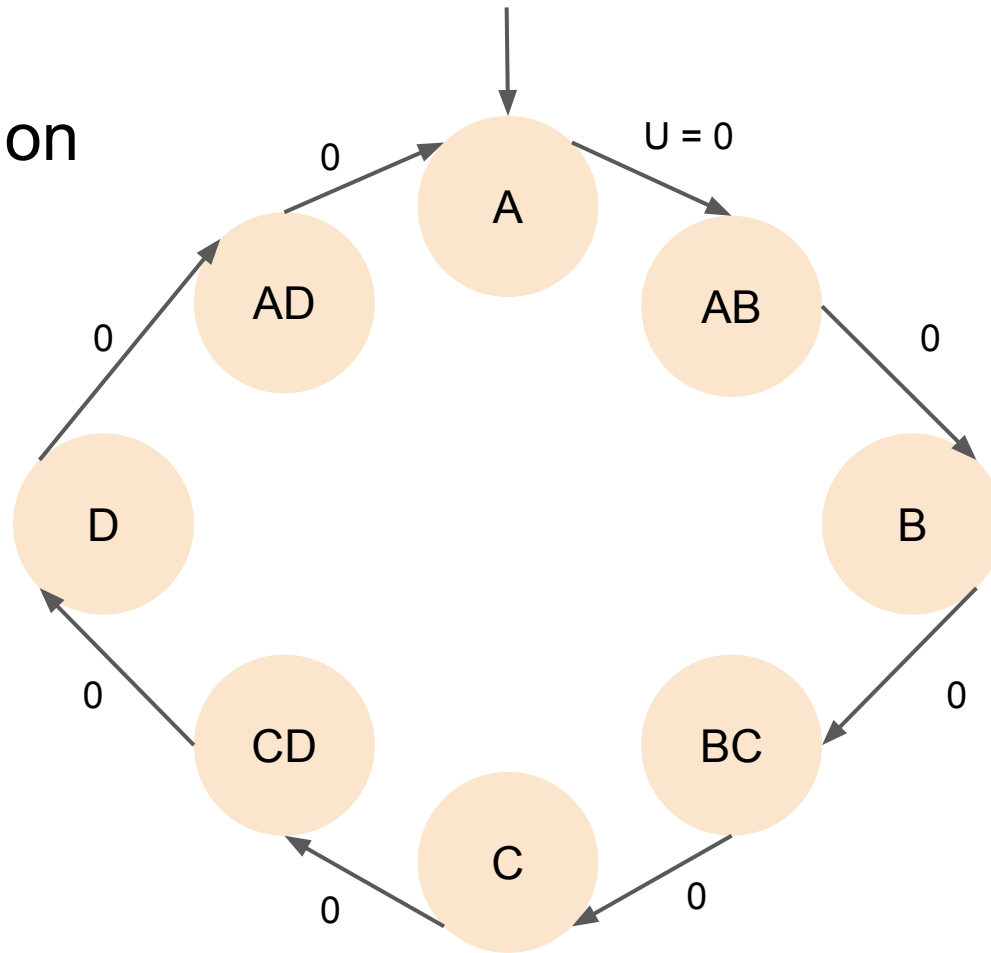


State transition diagram

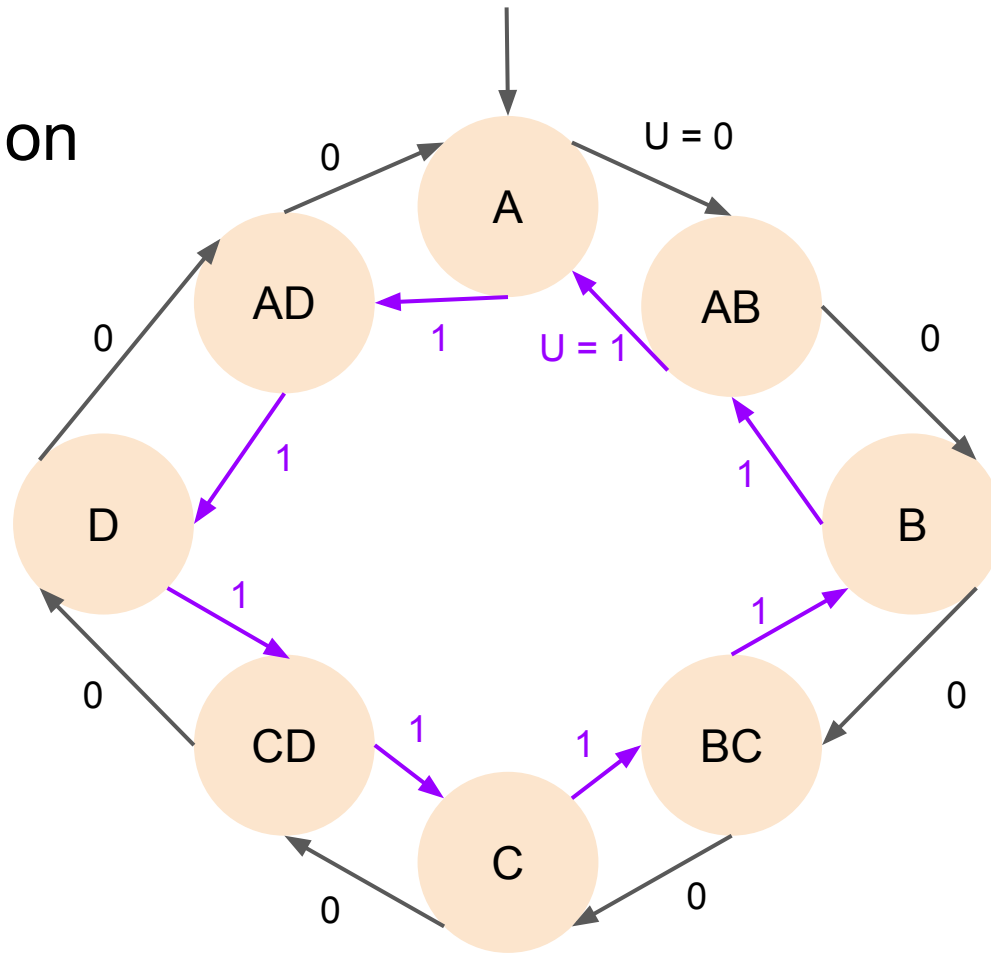


We can use only 3 bits to represent 8 states, but since we are working directly with boolean variables A, B, C, and D, it is actually more intuitive to use them as the states.

State transition diagram



State transition diagram

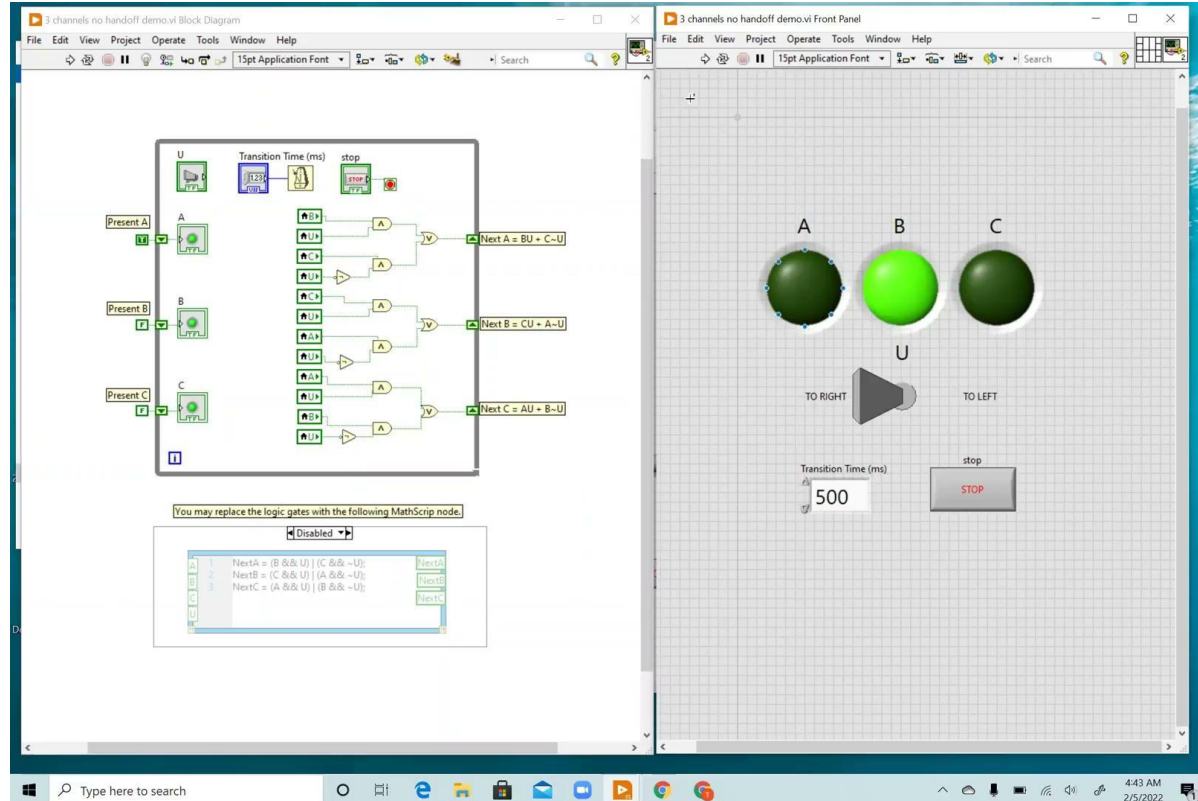


Implementation options

Option 1: Use the modulo (remainder) operator on the loop index i .

Option 2: Use shift registers and logic statements to determine what the next value should be based on the current values—**modeled as a Finite-State Machine (FSM)!**

Implementation: 3 channels no handoff demo.vi



VI Available
on Canvas

Example: 3 channels without handoff period

Example: Assume we have only three channels and no handoff period, and that $U=1$ (U is true) indicates clockwise motion. Then, to go clockwise, B should be set high in the following time step if A and U are high and C is low. We can write this as

$$B = A \& \sim C \& U, \text{ or just } A(\sim C)U$$

where $\sim$ indicates NOT and $\&$ indicates logical AND. If we are going counterclockwise, then B should be on if A and U are low and C is high, so the statement for B is

$$B = \sim A \& C \& \sim U.$$

This means B should turn on in either case, so we can write

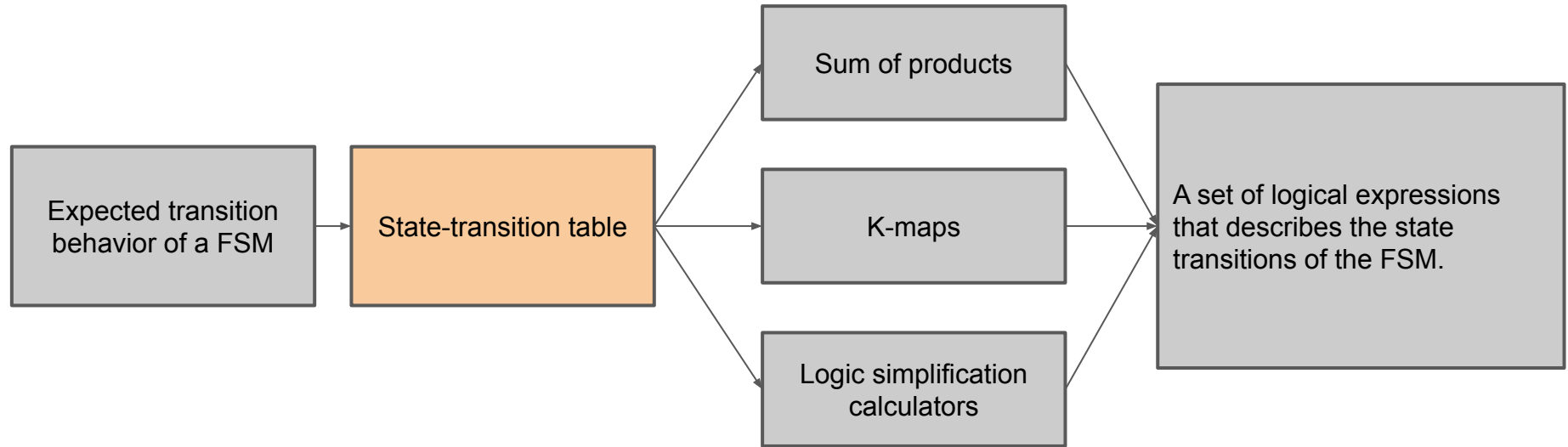
$$B = A \& \sim C \& U \parallel \sim A \& C \& \sim U, \text{ where } \parallel \text{ indicates logical OR.}$$

We can also use the notation for multiply (AND) and add (OR):

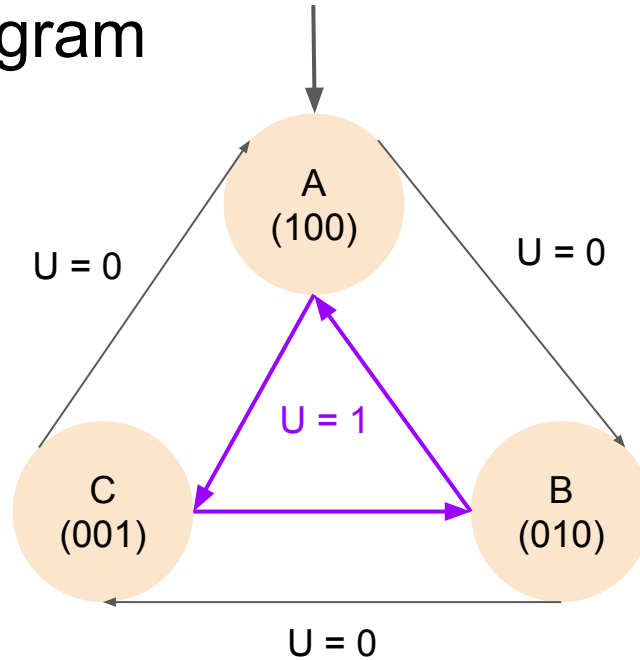
$$B = A(\sim C)U + \sim A C \sim U = A(\sim C)U + \sim A C (\sim U).$$

In this way, you can use Arduino code or a MathScript node to write your logical statements.

Systematic approaches to derive Finite State Machine expressions



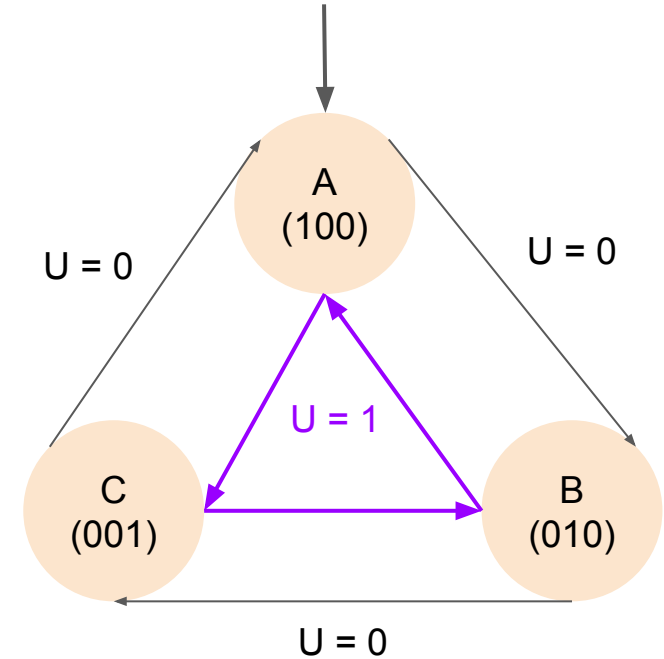
State transition diagram



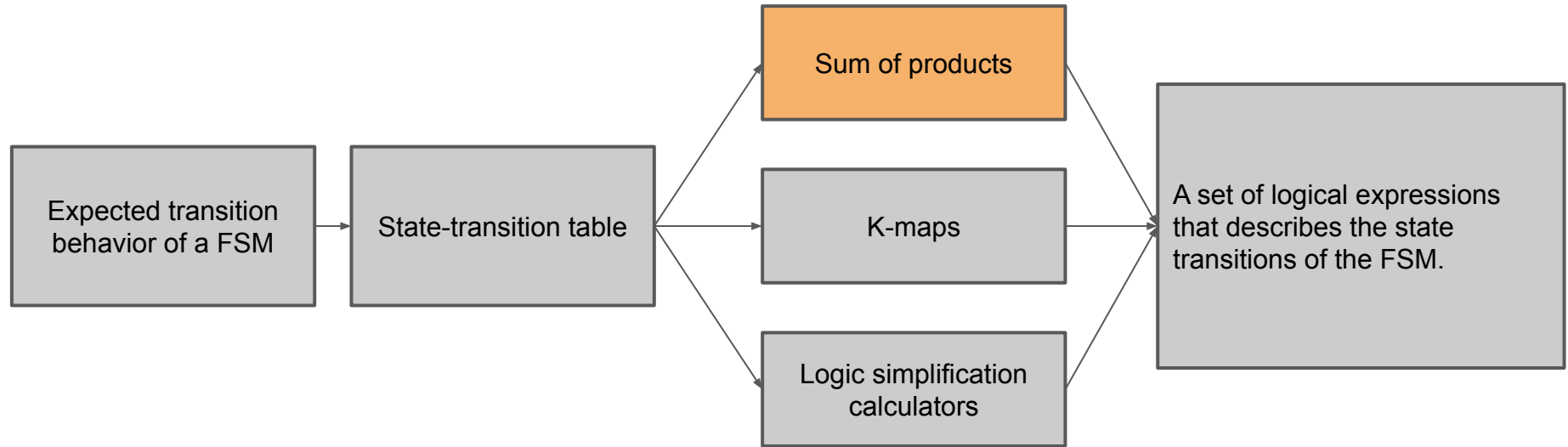
Although 2 bits are sufficient to represent 3 states, since we are working directly with boolean variables A, B, and C, it is actually more intuitive to use them as the states.

State transition table

Present State			Input	Next State		
A	B	C		A	B	C
1	0	0	0 (to right)	0	1	0
0	1	0		0	0	1
0	0	1		1	0	0
1	0	0	1 (to left)	0	0	1
0	1	0		1	0	0
0	0	1		0	1	0



Systematic approaches to derive Finite State Machine expressions



Sum of products

Present State			Input	Next State		
A	B	C		A	B	C
1	0	0	0 (to right)	0	1	0
0	1	0		0	0	1
0	0	1		1	0	0
1	0	0	1 (to left)	0	0	1
0	1	0		1	0	0
0	0	1		0	1	0

$\sim A \sim B C \sim U$

$\sim A B \sim C U$

$$\text{NextA} = \sim A \sim B C \sim U + \sim A B \sim C U$$

Sum of products

Present State			Input U	Next State		
A	B	C		A	B	C
1	0	0	0 (to right)	0	1	0
0	1	0		0	0	1
0	0	1		1	0	0
1	0	0	1 (to left)	0	0	1
0	1	0		1	0	0
0	0	1		0	1	0

$A \sim B \sim C \sim U$

$\sim A \sim B C U$

$$\text{NextB} = A \sim B \sim C \sim U + \sim A \sim B C U$$

Sum of products

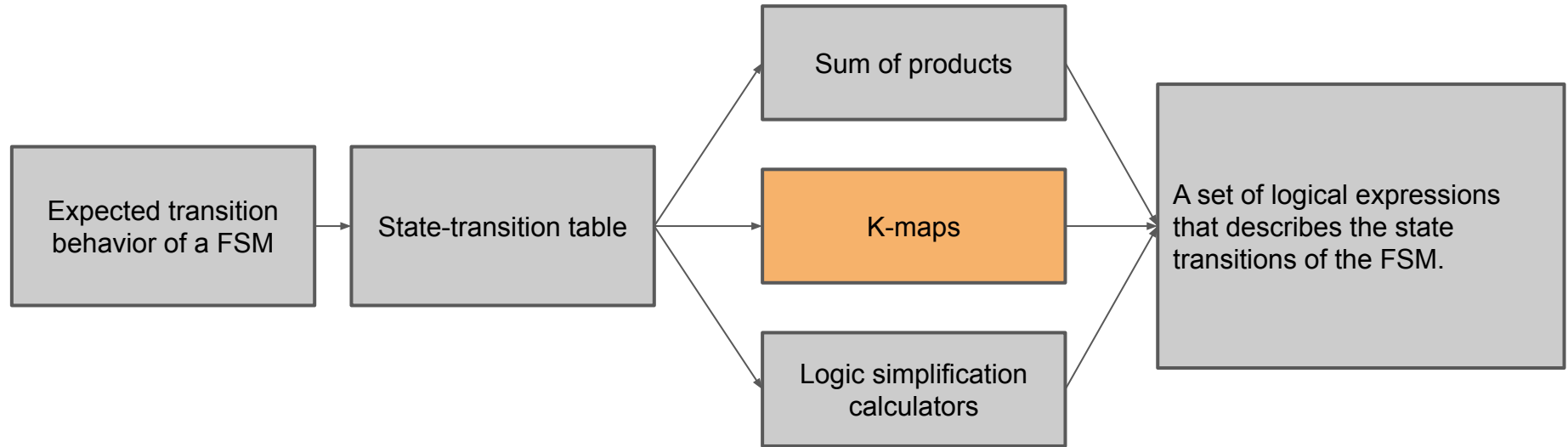
Present State			Input	Next State		
A	B	C		A	B	C
1	0	0	0 (to right)	0	1	0
0	1	0		0	0	1
0	0	1		1	0	0
1	0	0	1 (to left)	0	0	1
0	1	0		1	0	0
0	0	1		0	1	0

$\sim AB \sim C \sim U$

$A \sim B \sim CU$

$$\text{NextC} = \sim AB \sim C \sim U + A \sim B \sim CU$$

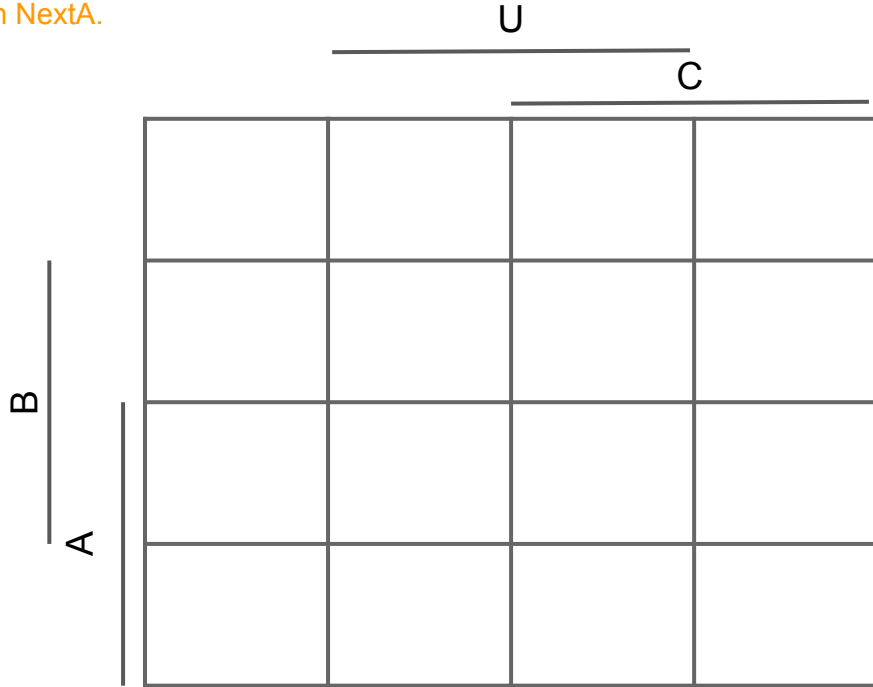
Systematic approaches to derive Finite State Machine expressions



Karnaugh maps (K-maps): For logic simplification

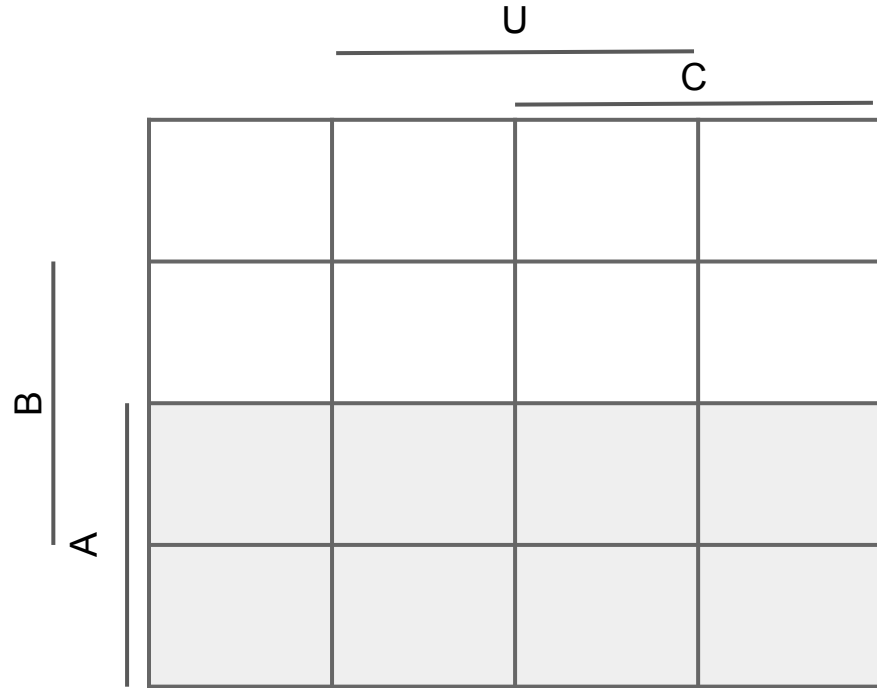
One next-state bit at a time. Let's start with NextA.

Present State			Input	Next State		
A	B	C		A	B	C
1	0	0	0	0	1	0
0	1	0		0	0	1
0	0	1		1	0	0
1	0	0	1	0	0	1
0	1	0		1	0	0
0	0	1		0	1	0



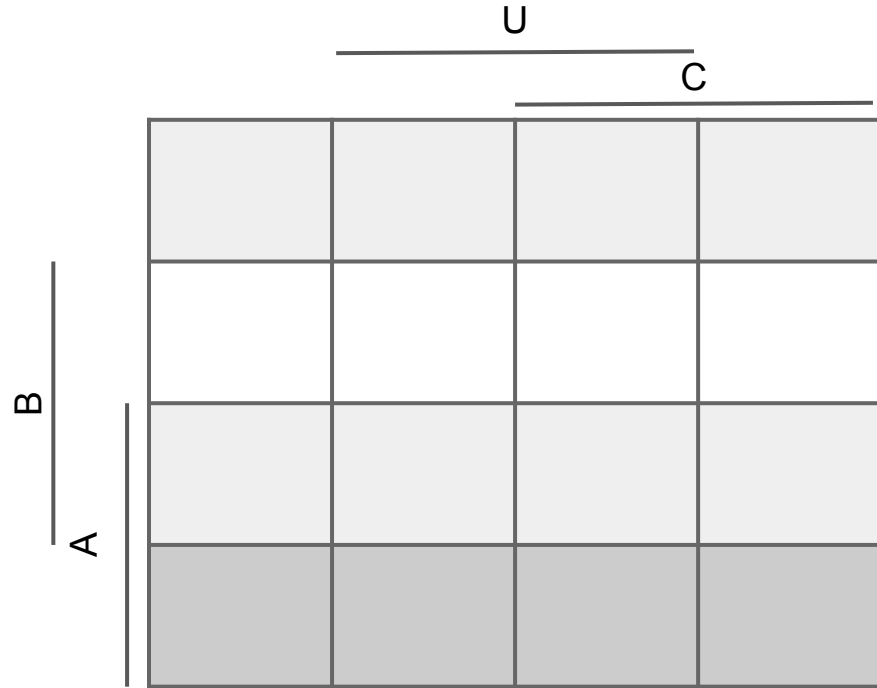
K-Map for NextA

Present State			Input	Next State		
A	B	C	U	A	B	C
1	0	0		0	1	0
0	1	0	0	0	0	1
0	0	1		1	0	0
1	0	0	1	0	0	1
0	1	0		1	0	0
0	0	1		0	1	0



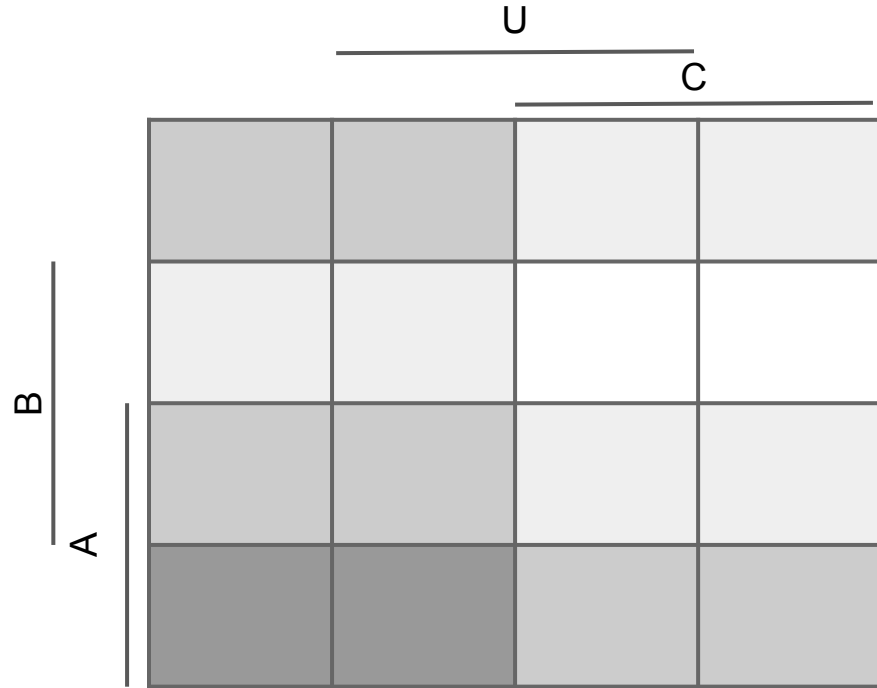
K-Map for NextA

Present State			Input	Next State		
A	B	C	U	A	B	C
1	0	0		0	1	0
0	1	0	0	0	0	1
0	0	1		1	0	0
1	0	0	1	0	0	1
0	1	0		1	0	0
0	0	1		0	1	0



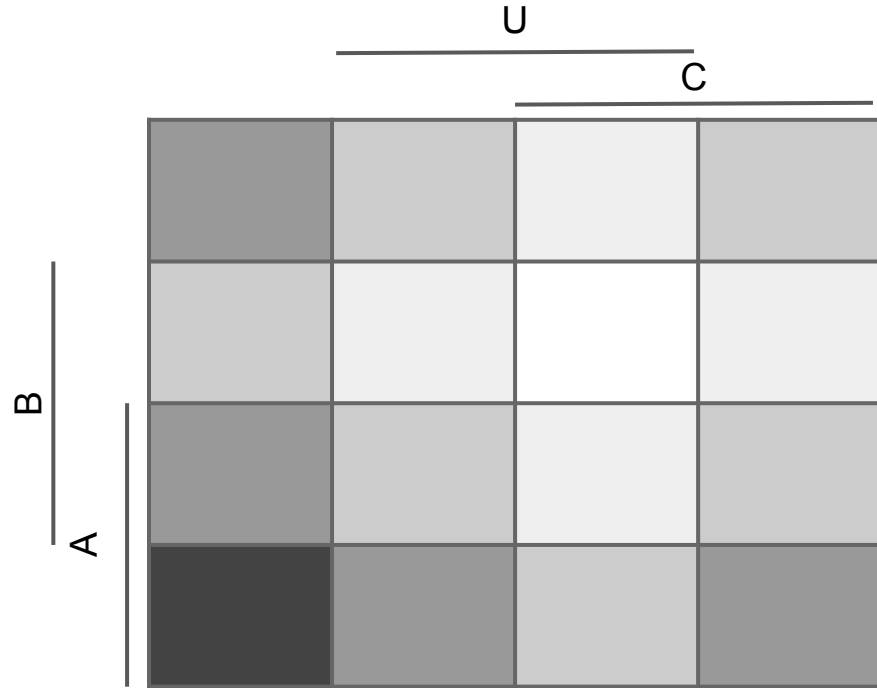
K-Map for NextA

Present State			Input U	Next State		
A	B	C		A	B	C
1	0	0		0	1	0
0	1	0	0	0	0	1
0	0	1		1	0	0
1	0	0	1	0	0	1
0	1	0		1	0	0
0	0	1		0	1	0



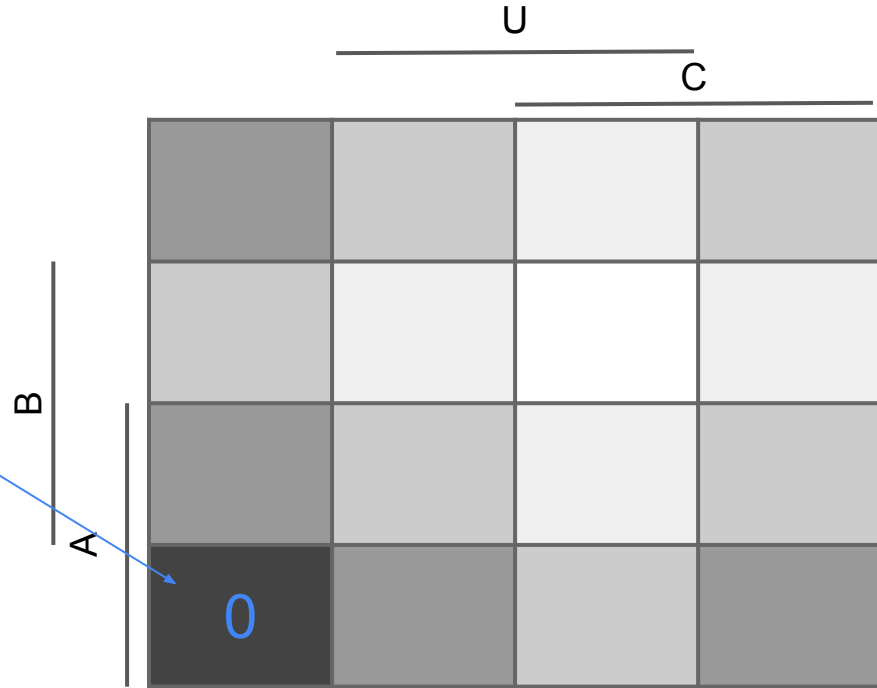
K-Map for NextA

Present State			Input	Next State		
A	B	C	U	A	B	C
1	0	0		0	1	0
0	1	0	0	0	0	1
0	0	1		1	0	0
1	0	0	1	0	0	1
0	1	0		1	0	0
0	0	1		0	1	0



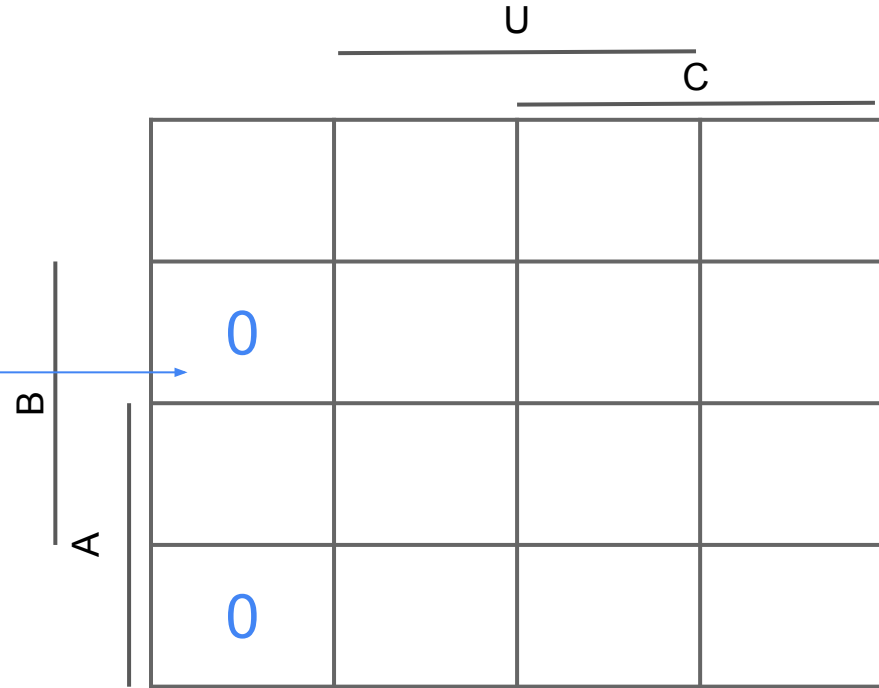
K-Map for NextA

Present State			Input	Next State		
A	B	C	U	A	B	C
1	0	0		0	1	0
0	1	0	0	0	0	1
0	0	1		1	0	0
1	0	0	1	0	0	1
0	1	0		1	0	0
0	0	1		0	1	0



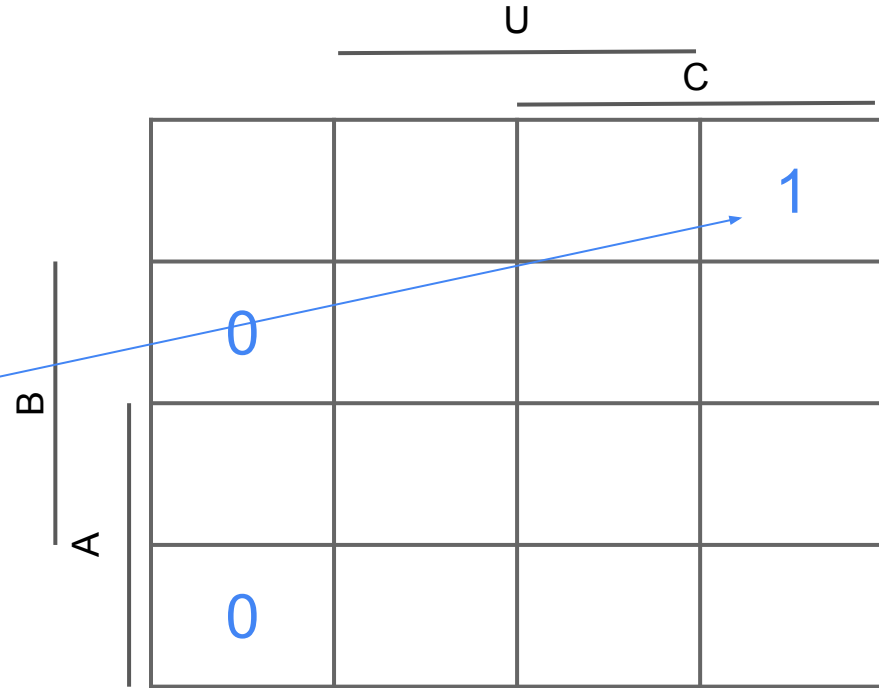
K-Map for NextA

Present State			Input	Next State		
A	B	C		A	B	C
1	0	0	1	0	1	0
0	1	0		0	0	1
0	0	1		1	0	0
1	0	0		0	0	1
0	1	0		1	0	0
0	0	1		0	1	0



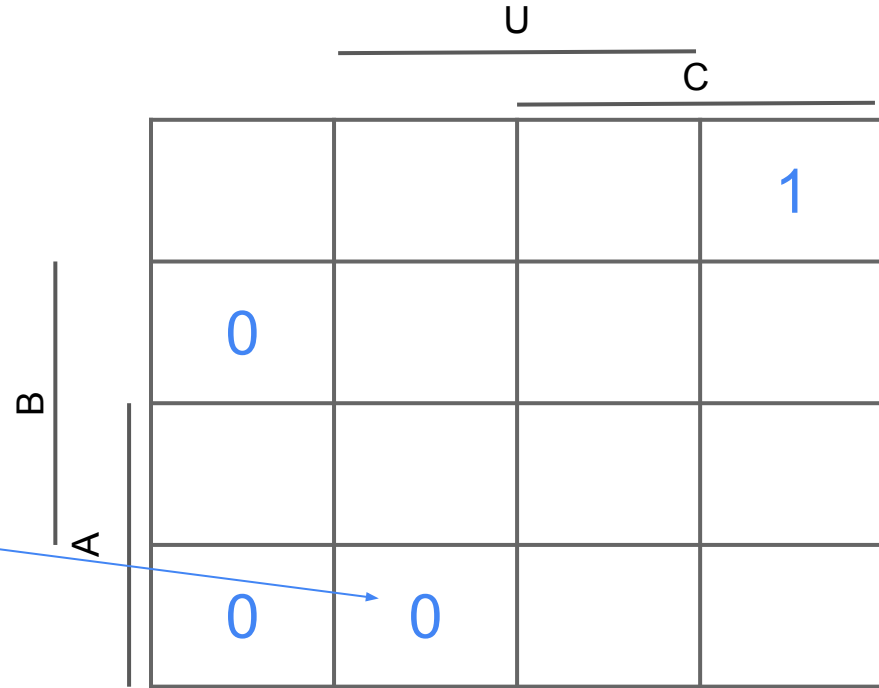
K-Map for NextA

Present State			Input	Next State		
A	B	C		A	B	C
1	0	0	0	0	1	0
0	1	0		0	0	1
0	0	1		1	0	0
1	0	0	1	0	0	1
0	1	0		1	0	0
0	0	1		0	1	0



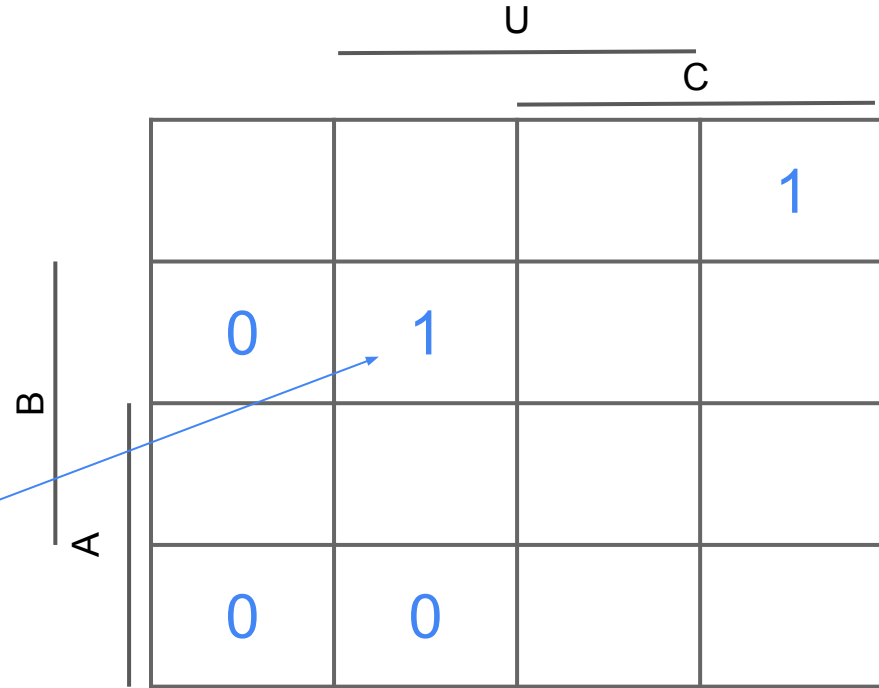
K-Map for NextA

Present State			Input	Next State		
A	B	C		A	B	C
1	0	0	0	0	1	0
0	1	0		0	0	1
0	0	1		1	0	0
1	0	0	1	0	0	1
0	1	0		1	0	0
0	0	1		0	1	0



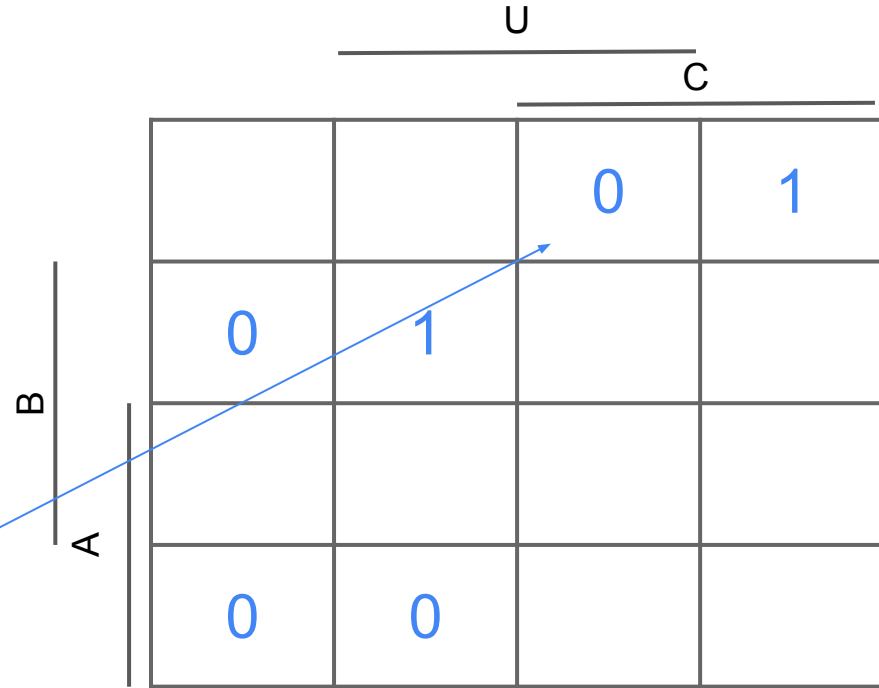
K-Map for NextA

Present State			Input U	Next State		
A	B	C		A	B	C
1	0	0	0	0	1	0
0	1	0		0	0	1
0	0	1		1	0	0
1	0	0	1	0	0	1
0	1	0		1	0	0
0	0	1		0	1	0



K-Map for NextA

Present State			Input	Next State		
A	B	C		A	B	C
1	0	0	0	0	1	0
0	1	0		0	0	1
0	0	1		1	0	0
1	0	0	1	0	0	1
0	1	0		1	0	0
0	0	1		0	1	0



K-Map for NextA

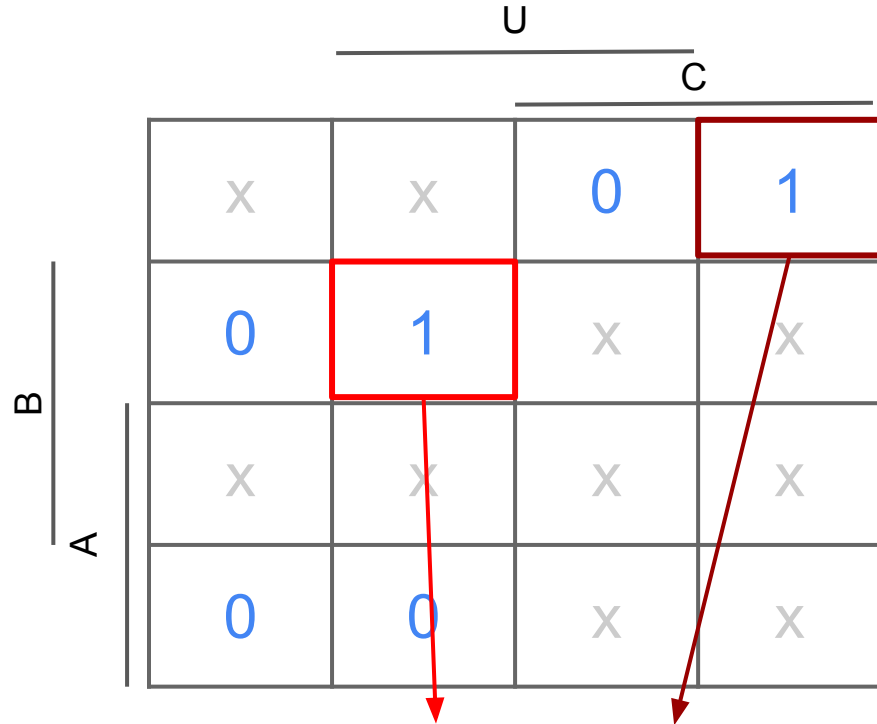
Present State			Input	Next State		
A	B	C		A	B	C
1	0	0	0	0	1	0
0	1	0		0	0	1
0	0	1		1	0	0
1	0	0	1	0	0	1
0	1	0		1	0	0
0	0	1		0	1	0

		U			
		C			
B	A	x	x	0	1
		0	1	x	x
		x	x	x	x
		0	0	x	x

X's = states we will never enter and thus ignored

K-Map for NextA

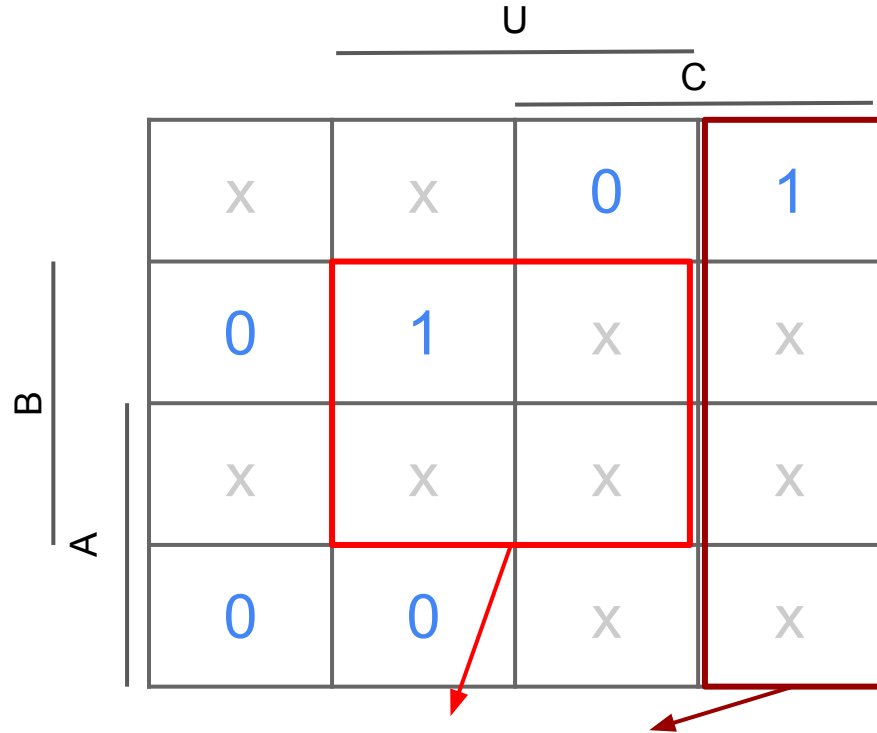
Present State			Input U	Next State		
A	B	C		A	B	C
1	0	0	0	0	1	0
0	1	0		0	0	1
0	0	1		1	0	0
1	0	0	1	0	0	1
0	1	0		1	0	0
0	0	1		0	1	0



Expression is too long. Can we simplify?

K-Map for NextA

Present State			Input U	Next State		
A	B	C		A	B	C
1	0	0	0	0	1	0
0	1	0		0	0	1
0	0	1		1	0	0
1	0	0	1	0	0	1
0	1	0		1	0	0
0	0	1		0	1	0



Larger red box = simpler expression.

K-Map for NextB

Let's do the same simplification for NextB.

Present State			Input U	Next State		
A	B	C		A	B	C
1	0	0	0	0	1	0
0	1	0		0	0	1
0	0	1		1	0	0
1	0	0	1	0	0	1
0	1	0		1	0	0
0	0	1		0	1	0

K maps are circular, so those two areas actually belong to the same box: A & ~U.

The Karnaugh map for NextB is a 4x4 grid with inputs A, B, C, and U. The map shows the next state of B (NextB) for each combination of inputs. The map is circular, meaning the top and bottom edges are connected, and the left and right edges are connected. Red boxes highlight two groups of 1s: one group (CU) and another group (A~U).

			U	
			C	

$$\text{NextB} = (CU) + (A\sim U)$$

K-Map for NextC

Let's do the same simplification for NextC.

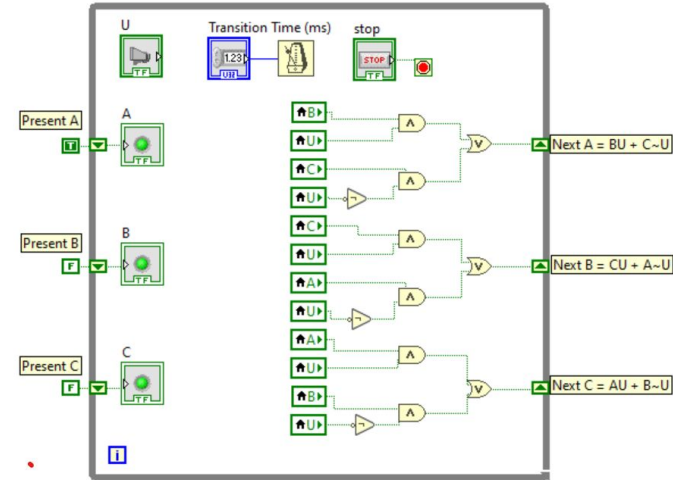
Present State			Input	Next State		
A	B	C		A	B	C
1	0	0	0	0	1	0
0	1	0		0	0	1
0	0	1		1	0	0
1	0	0	1	0	0	1
0	1	0		1	0	0
0	0	1		0	1	0

		U		
				C
	X	X	0	0
B	1	0	X	X
	X	X	X	X
A	0	1	X	X

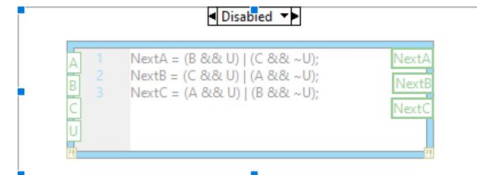
$$\text{NextC} = (AU) + (B\sim U)$$

3 channels without handoff: FSM expressions

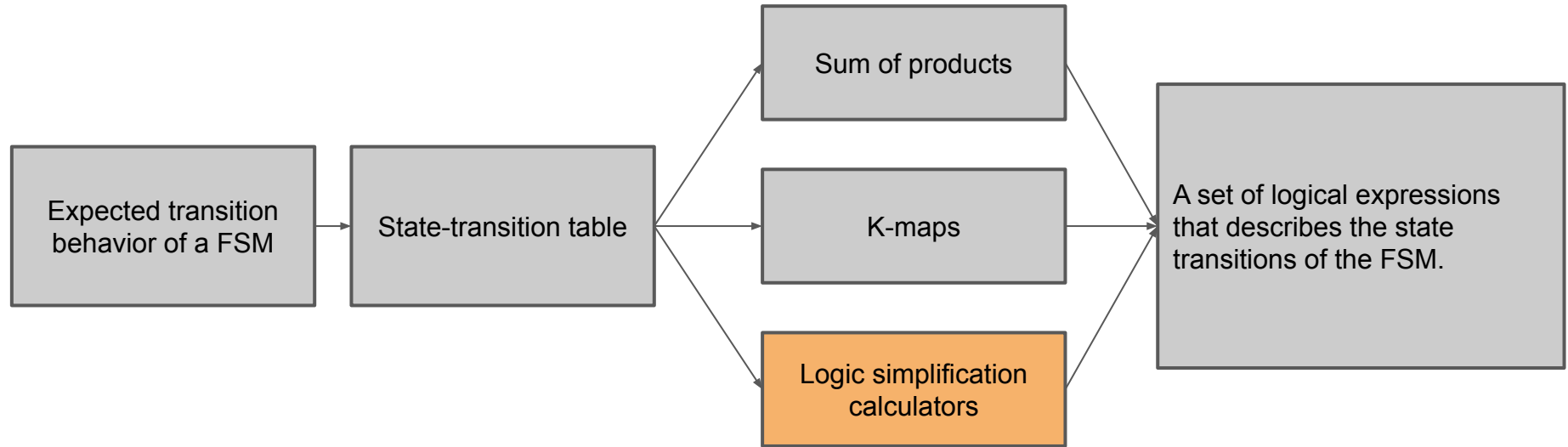
- $\text{NextA} = \text{BU} + \text{C}\sim\text{U}$
- $\text{NextB} = \text{CU} + \text{A}\sim\text{U}$
- $\text{NextC} = \text{AU} + \text{B}\sim\text{U}$



You may replace the logic gates with the following MathScript node.



Systematic approaches to derive Finite State Machine expressions

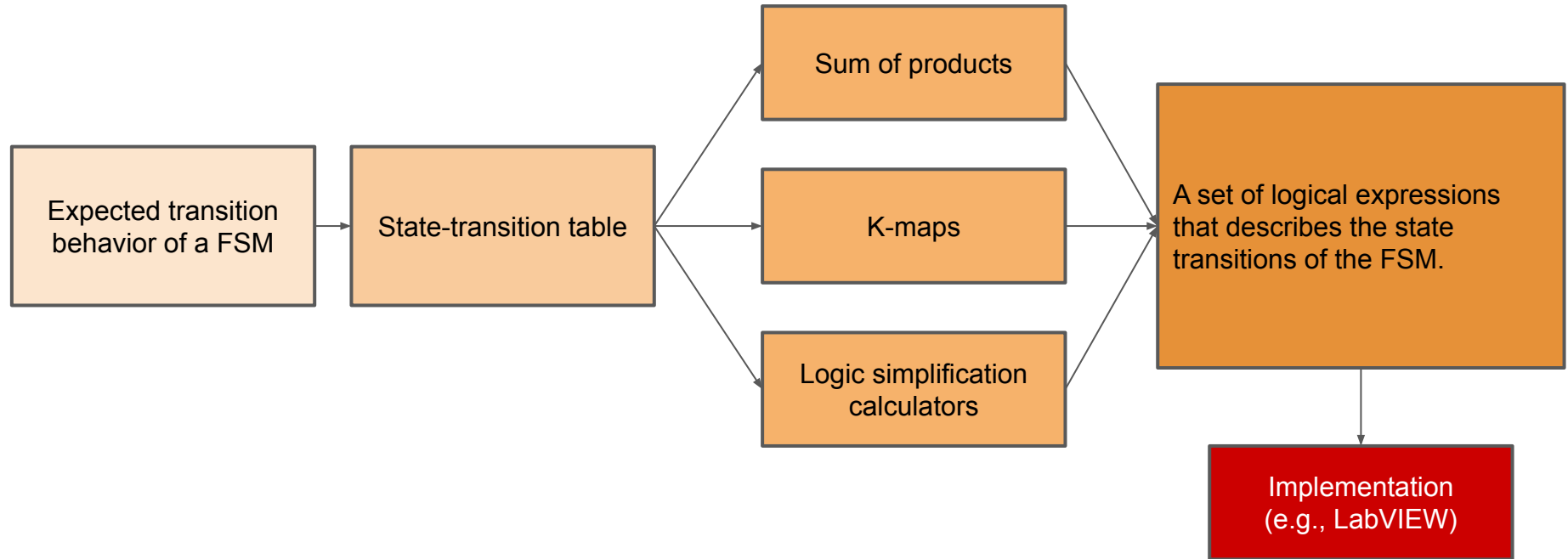


For lab 6: Logic simplification calculators

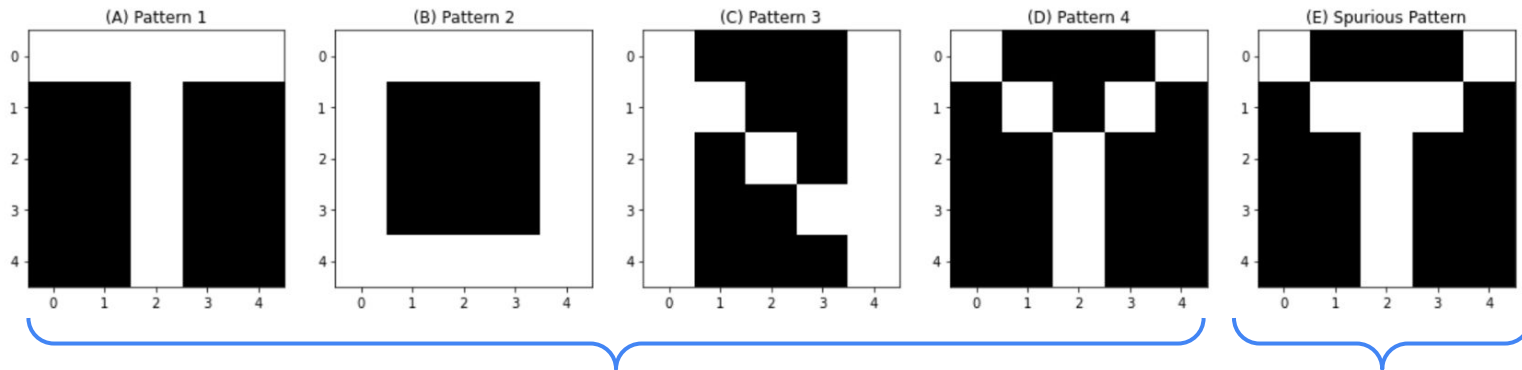
If you'd like to simplify the FSM expressions for lab 6 (i.e., 4 channels with handoff period), here's what to do:

1. Go to <http://www.32x8.com/index.html>
2. Fill out the 5-variable state-transition tables (one per next-state bit).
3. The website will automatically generate the logical expressions for you.

Systematic approaches to derive Finite State Machine expressions



Aside: Hopfield Model for Associative Memory



These 4 patterns are stored inside the connection matrix (J) of this Hopfield network.

An example spurious pattern

$$J_{ij} := \frac{1}{N} \sum_{\mu=1}^M p_i^{\mu} p_j^{\mu}.$$

Connection matrix derivation
(based on Hebbian learning)

$$S_i(t + \Delta t) := \text{sgn} \left[\sum_{j=1}^N J_{ij} S_j(t) \right].$$

Update rule
(Deterministic version)

Aside: Hopfield network demo

